

DriverAPI.inc的使用

Filament代码中，有多次DriverAPI.inc的引用，但每次引用位置所涉及到的功能又是不一样的：

```
$ grep -nr "DriverAPI.inc"|grep "\.h"
filament/backend/src/noop/NoopDriver.h:58:#include "private/backend/DriverAPI.inc"
filament/backend/src/CommandStreamDispatcher.h:70:#include "private/backend/DriverAPI.inc"
filament/backend/src/CommandStreamDispatcher.h:84:#include "private/backend/DriverAPI.inc"
filament/backend/src/vulkan/VulkanDriver.h:74:#include "private/backend/DriverAPI.inc"
filament/backend/src/metal/MetalDriver.h:86:#include "private/backend/DriverAPI.inc"
filament/backend/src/opengl/OpenGLDriver.h:224:#include "private/backend/DriverAPI.inc"
filament/backend/include/private/backend/CommandStream.h:243:#include "DriverAPI.inc"
filament/backend/include/private/backend/Dispatcher.h:45:#include "DriverAPI.inc"
filament/backend/include/private/backend/Driver.h:102:#include "private/backend/DriverAPI.inc"
```

下面就每个使用位置来进行分析。

在DriverAPI.inc文件内，最重要的就是声明了如下几个宏还有所有驱动API函数定义：

```
//
#define DECL_DRIVER_API_SYNCHRONOUS_N(R, N, ...) \
    DECL_DRIVER_API_SYNCHRONOUS(R, N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

#define DECL_DRIVER_API_SYNCHRONOUS_0(R, N) \
    DECL_DRIVER_API_SYNCHRONOUS(R, N,,)

DECL_DRIVER_API_SYNCHRONOUS_0(void, terminate)
DECL_DRIVER_API_SYNCHRONOUS_N(backend::StreamHandle, createStreamNative, void*, stream)

#define DECL_DRIVER_API_N(N, ...) \
    DECL_DRIVER_API_DECL_DRIVER_API_SYNCHRONOUSPI(N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

#define DECL_DRIVER_API_R_N(R, N, ...) \
    DECL_DRIVER_API_RETURN(R, N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

#define PARAM_1( P, ...) P
#define ARG_1( T, P, ...) T P
#define DECL_DRIVER_API_0(N) DECL_DRIVER_API(N, ARG_1(int, dummy=0), PARAM_1(dummy))
#define DECL_DRIVER_API_R_0(R, N) DECL_DRIVER_API_RETURN(R, N, ARG_1(int, dummy=0), PARAM_1(dummy))

//,DriverAPI
...
DECL_DRIVER_API_0(flush)
DECL_DRIVER_API_N(beginFrame,
    int64_t, monotonic_clock_ns,
    uint32_t, frameId)
...
DECL_DRIVER_API_R_0(backend::FenceHandle, createFence)
...
DECL_DRIVER_API_R_N(backend::IndexBufferHandle, createIndexBuffer,
    backend::ElementType, elementType,
    uint32_t, indexCount,
    backend::BufferUsage, usage)
...
DECL_DRIVER_API_SYNCHRONOUS_0(void, terminate)
DECL_DRIVER_API_SYNCHRONOUS_N(backend::StreamHandle, createStreamNative, void*, stream)
...
```

也就是如下六个核心的宏定义，这些宏定义最终也是为定义后面驱动API函数来服务的：

```
#define DECL_DRIVER_API_SYNCHRONOUS_N //
#define DECL_DRIVER_API_SYNCHRONOUS_0 //

#define DECL_DRIVER_API_N //
#define DECL_DRIVER_API_0 //

#define DECL_DRIVER_API_R_N //
#define DECL_DRIVER_API_R_0 //
```

而从上面DriverAPI.inc中可以看到，这六个宏定义又依赖于下面三个宏定义：

```
#define DECL_DRIVER_API_SYNCHRONOUS //
#define DECL_DRIVER_API //
#define DECL_DRIVER_API_RETURN //
```

通过DriverAPI.inc中注释可以看到：

```
/*
 * How to use this header file.
 *
 * The DECL_DRIVER_API() macro must be defined before including this file:
 *   DECL_DRIVER_API(method_name, params_decl, params)
 *
 *   method_name      the name of the method (e.g. beginRenderPass)
 *   params_decl      the list of typed params
 *   params           the parameters, without their type (e.g.: for a function call)
 *
 * The following example adds all the driver methods as virtual
 * members of class:
 *
 * class MyDriver : public Driver {
 * public:
 *   #define DRIVER_API(method_name, params_decl, params) \
 *     virtual void method_name(params_decl) override
 *   #include "DriverAPI.h"
 * };
 *
 * DECL_DRIVER_API is automatically undefined.
 */
```

我们需要在使用这个DriverAPI.inc文件之前，需要定义DECL_DRIVER_API这个宏。从上面注释中的例子来看，通过宏定义，将所有driver方法添加到MyDriver类的虚函数。也就是这个文件就是为了用简单方式统一处理所有驱动方法。

下面就依次分析上面搜索结果下每个引入DriverAPI.inc文件的具体意义：

```
//Dispatcher.h

class Dispatcher {
public:
using Execute = void (*)(Driver& driver, CommandBase* self, intptr_t* next);
#define DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params)
#define DECL_DRIVER_API(methodName, paramsDecl, params) Execute methodName##_;
#define DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) Execute methodName##_;

#include "DriverAPI.inc"
};
```

从上面定义看：

DECL_DRIVER_API_SYNCHRONOUS被定义为空，也就是由DECL_DRIVER_API_SYNCHRONOUS定义的类型会被预处理器忽略掉。

DECL_DRIVER_API被定义为：

```
Execute methodName_;
```

DECL_DRIVER_API_RETURN被定义为:

```
Execute methodName_;
```

也就是定义了如下内容:

```
//Dispatcher.h

class Dispatcher {
public:
using Execute = void (*)(Driver& driver, CommandBase* self, intptr_t* next);
    Execute flush_;
    Execute beginFrame_;
    Execute createFence_;
    Execute createIndexBuffer_;
};
```

可以看到, 已经按上面所说, DECL_DRIVER_API_SYNCHRONOUS定义的terminate和createStreamNative都被忽略掉。

如何验证我们所说的是否正确呢?

我们可以通过将源程序编译过程做处理, 只对源文件进行预处理, 而不做后续处理, 这样, 就能看到我们预处理的结果, 也就是可以验证我们所说的结果, 我们首先写一个测试代码, 将DriverAPI内部:

```
//test-macro.cpp

#include <iostream>

//DriverAPI.inc
#define DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params)
#define DECL_DRIVER_API(methodName, paramsDecl, params)          Execute methodName##_;
#define DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) Execute methodName##_;

#define PAIR_ARGS_0(M, ...)
#define PAIR_ARGS_1(M, X, Y, ...) M(X, Y)
#define PAIR_ARGS_2(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_1(M, __VA_ARGS__))
#define PAIR_ARGS_3(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_2(M, __VA_ARGS__))
#define PAIR_ARGS_4(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_3(M, __VA_ARGS__))
#define PAIR_ARGS_5(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_4(M, __VA_ARGS__))
#define PAIR_ARGS_6(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_5(M, __VA_ARGS__))
#define PAIR_ARGS_7(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_6(M, __VA_ARGS__))
#define PAIR_ARGS_8(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_7(M, __VA_ARGS__))
#define PAIR_ARGS_9(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_8(M, __VA_ARGS__))
#define PAIR_ARGS_10(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_9(M, __VA_ARGS__))
#define PAIR_ARGS_11(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_10(M, __VA_ARGS__))
#define PAIR_ARGS_12(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_11(M, __VA_ARGS__))

#define PAIR_ARGS_N__(_0, E1, _1, E2, _2, E3, _3, E4, _4, E5, _5, E6, _6, E7, _7, E8, _8, E9, _9, E10, _10, E11, _11, E12, _12, X, ...) PAIR_ARGS_##X

#define PAIR_ARGS_N(M, ...) \
    EXPAND(EXPAND(PAIR_ARGS_N__(0, ##__VA_ARGS__, 12, E, 11, E, 10, E, 9, E, 8, E, 7, E, 6, E, 5, E, 4, E, 3, E, 2, E, 1, E, 0))(M, __VA_ARGS__))

#define ARG(T, P) T P

#define PARAM(T, P) P

#define DECL_DRIVER_API_N(N, ...) \
    DECL_DRIVER_API(N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

#define DECL_DRIVER_API_R_N(R, N, ...) \
    DECL_DRIVER_API_RETURN(R, N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))
```

```

#define DECL_DRIVER_API_SYNCHRONOUS_N(R, N, ...) \
    DECL_DRIVER_API_SYNCHRONOUS(R, N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

// on some compilers the ##__VA_ARGS__ hack is not supported, so we can't handle 0-parameter APIs
// with DECL_DRIVER_API_SYNCHRONOUS_N
#define DECL_DRIVER_API_SYNCHRONOUS_0(R, N) \
    DECL_DRIVER_API_SYNCHRONOUS(R, N,,)

#define PARAM_1( P, ...) P
#define ARG_1( T, P, ...) T P
#define DECL_DRIVER_API_0(N)      DECL_DRIVER_API(N, ARG_1(int, dummy=0), PARAM_1(dummy))
#define DECL_DRIVER_API_R_0(R, N) DECL_DRIVER_API_RETURN(R, N, ARG_1(int, dummy=0), PARAM_1(dummy))

class Dispatcher
{
    using Execute = void (*)(int arg1, int arg2, intptr_t* next);

    DECL_DRIVER_API_0(flush)
    DECL_DRIVER_API_N(beginFrame,
                      int64_t, monotonic_clock_ns,
                      uint32_t, frameId)
    DECL_DRIVER_API_R_0(backend::FenceHandle, createFence)
    DECL_DRIVER_API_R_N(backend::IndexBufferHandle, createIndexBuffer,
                      backend::ElementType, elementType,
                      uint32_t, indexCount,
                      backend::BufferUsage, usage)
    DECL_DRIVER_API_SYNCHRONOUS_0(void, terminate)
    DECL_DRIVER_API_SYNCHRONOUS_N(backend::StreamHandle, createStreamNative, void*, stream)
};

int main()
{
    return 0;
}

```

然后，我们编译代码，只做预处理：

```
g++ -E test-macro.cpp -o test-macro.i
```

然后，我们打开test-macro.i文件，我们可以看到，最终我们的预处理结果：

```

//test-macro.i

class Dispatcher
{
    using Execute = void (*)(int arg1, int arg2, intptr_t* next);

    Execute flush_;
    Execute beginFrame_;
    Execute createFence_;
    Execute createIndexBuffer_;
};

int main()
{
    return 0;
}

```

我们看到，结果确实和我们预期是一致的。

下面，我们再看Driver.h：

```

class Driver {
public:
...

    /*
     * Asynchronous calls here only to provide a type to CommandStream. They must be non-virtual
     * so that calling the concrete implementation won't go through a vtable.
     *
     * Synchronous calls are virtual and are called directly by CommandStream.
     */

#define DECL_DRIVER_API(methodName, paramsDecl, params) \
    void methodName(paramsDecl) {}

#define DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params) \
    virtual RetType methodName(paramsDecl) = 0;

#define DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) \
    virtual RetType methodName##S() noexcept = 0; \
    void methodName##R(RetType, paramsDecl) {}

#include "private/backend/DriverAPI.inc"
};

```

从上面注释中，可以看到，这里的异步调用是为CommandStream提供了一个类型，调用一定是非虚的方法，以便在调用实际实现的时候能够不经过虚表，而同步的调用是虚的，并且由CommandStream来直接调用。

也就是，这里的DECL_DRIVER_API(methodName, paramsDecl, params)会被定义为：

```
void methodName(paramsDecl) {}
```

而DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params)被定义为：

```
virtual RetType methodName(paramsDecl) = 0;
```

DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params)被定义为：

```
virtual RetType methodName##S() noexcept = 0; \
void methodName##R(RetType, paramsDecl) {}
```

代入我们上面的DriverAPI：

```

//,DriverAPI
...
DECL_DRIVER_API_0(flush)
DECL_DRIVER_API_N(beginFrame,
    int64_t, monotonic_clock_ns,
    uint32_t, frameId)
...
DECL_DRIVER_API_R_0(backend::FenceHandle, createFence)
...
DECL_DRIVER_API_R_N(backend::IndexBufferHandle, createIndexBuffer,
    backend::ElementType, elementType,
    uint32_t, indexCount,
    backend::BufferUsage, usage)
...
DECL_DRIVER_API_SYNCHRONOUS_0(void, terminate)
DECL_DRIVER_API_SYNCHRONOUS_N(backend::StreamHandle, createStreamNative, void*, stream)
...

```

这六个API就会被预处理为：

```

void flush(int dummy=0) {}
void beginFrame(int64_t monotonic_clock_ns, uint32_t frameId) {}

virtual backend::FenceHandle createFenceS() noexcept = 0;
void createFenceR(backend::FenceHandle, int dummy=0) {}

virtual backend::IndexBufferHandle createIndexBuffers() noexcept = 0;
void createIndexBufferR(backend::IndexBufferHandle, backend::ElementType elementType, uint32_t indexCount,
backend::BufferUsage usage) {}

virtual void terminate() = 0;
virtual backend::StreamHandle createStreamNative(void* stream) = 0;

```

也就是Driver.h中的Driver类会添加如下定义:

```

//Driver.h

class Driver {
public:
...
    void flush(int dummy=0) {}
    void beginFrame(int64_t monotonic_clock_ns, uint32_t frameId) {}

    virtual backend::FenceHandle createFenceS() noexcept = 0;
    void createFenceR(backend::FenceHandle, int dummy=0) {}

    virtual backend::IndexBufferHandle createIndexBuffers() noexcept = 0;
    void createIndexBufferR(backend::IndexBufferHandle, backend::ElementType elementType, uint32_t
indexCount, backend::BufferUsage usage) {}

    virtual void terminate() = 0;
    virtual backend::StreamHandle createStreamNative(void* stream) = 0;
...
};

```

同样, 我们可以继续写测试程序验证:

```

//test-macro2.cpp

#include <iostream>

//DriverAPI.inc
#define DECL_DRIVER_API(methodName, paramsDecl, params) \
    void methodName(paramsDecl) {}

#define DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params) \
    virtual RetType methodName(paramsDecl) = 0;

#define DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) \
    virtual RetType methodName##S() noexcept = 0; \
    void methodName##R(RetType, paramsDecl) {}

#define EXPAND(x) x

#define APPLY0(M, ...)
#define APPLY1(M, A, ...) EXPAND(M(A))
#define APPLY2(M, A, ...) EXPAND(M(A)), EXPAND(APPLY1(M, __VA_ARGS__))
#define APPLY3(M, A, ...) EXPAND(M(A)), EXPAND(APPLY2(M, __VA_ARGS__))
#define APPLY4(M, A, ...) EXPAND(M(A)), EXPAND(APPLY3(M, __VA_ARGS__))
#define APPLY5(M, A, ...) EXPAND(M(A)), EXPAND(APPLY4(M, __VA_ARGS__))
#define APPLY6(M, A, ...) EXPAND(M(A)), EXPAND(APPLY5(M, __VA_ARGS__))
#define APPLY7(M, A, ...) EXPAND(M(A)), EXPAND(APPLY6(M, __VA_ARGS__))
#define APPLY8(M, A, ...) EXPAND(M(A)), EXPAND(APPLY7(M, __VA_ARGS__))
#define APPLY9(M, A, ...) EXPAND(M(A)), EXPAND(APPLY8(M, __VA_ARGS__))
#define APPLY10(M, A, ...) EXPAND(M(A)), EXPAND(APPLY9(M, __VA_ARGS__))

```

```

#define APPLY11(M, A, ...) EXPAND(M(A)), EXPAND(APPLY10(M, __VA_ARGS__))
#define APPLY12(M, A, ...) EXPAND(M(A)), EXPAND(APPLY11(M, __VA_ARGS__))
#define APPLY_N(_0, _1, _2, _3, _4, _5, _6, _7, _8, _9, _10, _11, _12, X, ...) APPLY##X
#define APPLY(M, ...) EXPAND(EXPAND(APPLY_N(M, __VA_ARGS__, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0))(M,
__VA_ARGS__))

#define PAIR_ARGS_0(M, ...)
#define PAIR_ARGS_1(M, X, Y, ...) M(X, Y)
#define PAIR_ARGS_2(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_1(M, __VA_ARGS__))
#define PAIR_ARGS_3(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_2(M, __VA_ARGS__))
#define PAIR_ARGS_4(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_3(M, __VA_ARGS__))
#define PAIR_ARGS_5(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_4(M, __VA_ARGS__))
#define PAIR_ARGS_6(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_5(M, __VA_ARGS__))
#define PAIR_ARGS_7(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_6(M, __VA_ARGS__))
#define PAIR_ARGS_8(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_7(M, __VA_ARGS__))
#define PAIR_ARGS_9(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_8(M, __VA_ARGS__))
#define PAIR_ARGS_10(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_9(M, __VA_ARGS__))
#define PAIR_ARGS_11(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_10(M, __VA_ARGS__))
#define PAIR_ARGS_12(M, X, Y, ...) M(X, Y), EXPAND(PAIR_ARGS_11(M, __VA_ARGS__))

#define PAIR_ARGS_N(_0, E1, _1, E2, _2, E3, _3, E4, _4, E5, _5, E6, _6, E7, _7, E8, _8, E9, _9, E10, _10,
E11, _11, E12, _12, X, ...) PAIR_ARGS_##X

#define PAIR_ARGS_N(M, ...) \
    EXPAND(EXPAND(PAIR_ARGS_N(_0, ##__VA_ARGS__, 12, E, 11, E, 10, E, 9, E, 8, E, 7, E, 6, E, 5, E, 4, E, 3,
E, 2, E, 1, E, 0))(M, __VA_ARGS__))

#define ARG(T, P) T P

#define PARAM(T, P) P

#define DECL_DRIVER_API_N(N, ...) \
    DECL_DRIVER_API(N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

#define DECL_DRIVER_API_R_N(R, N, ...) \
    DECL_DRIVER_API_RETURN(R, N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

#define DECL_DRIVER_API_SYNCHRONOUS_N(R, N, ...) \
    DECL_DRIVER_API_SYNCHRONOUS(R, N, PAIR_ARGS_N(ARG, ##__VA_ARGS__), PAIR_ARGS_N(PARAM, ##__VA_ARGS__))

// on some compilers the ##__VA_ARGS__ hack is not supported, so we can't handle 0-parameter APIs
// with DECL_DRIVER_API_SYNCHRONOUS_N
#define DECL_DRIVER_API_SYNCHRONOUS_0(R, N) \
    DECL_DRIVER_API_SYNCHRONOUS(R, N,,)

#define PARAM_1( P, ...) P
#define ARG_1( T, P, ...) T P
#define DECL_DRIVER_API_0(N) DECL_DRIVER_API(N, ARG_1(int, dummy=0), PARAM_1(dummy))
#define DECL_DRIVER_API_R_0(R, N) DECL_DRIVER_API_RETURN(R, N, ARG_1(int, dummy=0), PARAM_1(dummy))

class Driver
{
    DECL_DRIVER_API_0(flush)
    DECL_DRIVER_API_N(beginFrame,
        int64_t, monotonic_clock_ns,
        uint32_t, frameId)
    DECL_DRIVER_API_R_0(backend::FenceHandle, createFence)
    DECL_DRIVER_API_R_N(backend::IndexBufferHandle, createIndexBuffer,
        backend::ElementType, elementType,
        uint32_t, indexCount,
        backend::BufferUsage, usage)
    DECL_DRIVER_API_SYNCHRONOUS_0(void, terminate)
    DECL_DRIVER_API_SYNCHRONOUS_N(backend::StreamHandle, createStreamNative, void*, stream)
};

int main()
{
    return 0;
}

```

同样的方式预处理：

```
g++ -E test-macro2.cpp -o test-macro2.i
```

我们查看test-macro2.i文件检查结果，和预期一致。

下面，我们再看CommandStream.h：

```
//CommandStream.h

class CommandStream {
...
public:
#define DECL_DRIVER_API(methodName, paramsDecl, params) \
    inline void methodName(paramsDecl) { \
        DEBUG_COMMAND_BEGIN(methodName, false, params); \
        using Cmd = COMMAND_TYPE(methodName); \
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd))); \
        new(p) Cmd(mDispatcher.methodName##_ , APPLY(std::move, params)); \
        DEBUG_COMMAND_END(methodName, false); \
    }

#define DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params) \
    inline RetType methodName(paramsDecl) { \
        DEBUG_COMMAND_BEGIN(methodName, true, params); \
        AutoExecute callOnExit([=]() { \
            DEBUG_COMMAND_END(methodName, true); \
        }); \
        return apply(&Driver::methodName, mDriver, std::forward_as_tuple(params)); \
    }

#define DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) \
    inline RetType methodName(paramsDecl) { \
        DEBUG_COMMAND_BEGIN(methodName, false, params); \
        RetType result = mDriver.methodName##S(); \
        using Cmd = COMMAND_TYPE(methodName##R); \
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd))); \
        new(p) Cmd(mDispatcher.methodName##_ , RetType(result), APPLY(std::move, params)); \
        DEBUG_COMMAND_END(methodName, false); \
        return result; \
    }

#include "DriverAPI.inc"
...
};
```

这里DEBUG_COMMAND_BEGIN和DEBUG_COMMAND_END是用来调试的选项，未开启状态下会被预处理器替换为空，暂时不管。那么DECL_DRIVER_API会被定义为：

```
inline void methodName(paramsDecl) {
    using Cmd = COMMAND_TYPE(methodName);
    void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
    new(p) Cmd(mDispatcher.methodName_, APPLY(std::move, params));
}
```

而COMMAND_TYPE则定义为：

```
#define COMMAND_TYPE(method) CommandType<decltype(&Driver::method)>::Command<&Driver::method>
```

也就是说DECL_DRIVER_API会变成：


```
inline void methodName(paramsDecl) {
    using Cmd = CommandType<decltype(&Driver::methodName)>::Command<&Driver::methodName>
    void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
    new(p) Cmd(mDispatcher.methodName_, APPLY(std::move, params));
}
```

这也就是上面Driver.h注释中说的“异步调用是为CommandStream提供了一个类型”，就是decltype所取得的Driver::methodName方法本身所属类型作为模板参数来定义的CommandType::Command类。

通过placement new来创建一个Cmd，并通过mDispatcher.methodName_和std::move过来的参数来初始化Cmd。

DECL_DRIVER_API_SYNCHRONOUS则被定义为：

```
inline RetType methodName(paramsDecl) {
    AutoExecute callOnExit([=]() {
    });
    return apply(&Driver::methodName, mDriver, std::forward_as_tuple(params));
}
```

DECL_DRIVER_API_RETURN被定义为：

```
inline RetType methodName(paramsDecl) {
    RetType result = mDriver.methodNameS();
    using Cmd = CommandType<decltype(&Driver::methodNameR)>::Command<&Driver::methodNameR>
    void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
    new(p) Cmd(mDispatcher.methodName_, RetType(result), APPLY(std::move, params));
    return result;
}
```

也就是CommandType::Command会以Driver::methodNameR作为模板参数来创建Cmd。

将DriverAPI.inc内函数定义代入CommandStream类中，最终预处理后代码就变成：

```

//CommandStream.h

class CommandStream
{
...
    inline void flush(int dummy=0) {
        using Cmd = CommandType<decltype(&Driver::flush)>::Command<&Driver::flush>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.flush_, std::move(dummy));
    }

    inline void beginFrame(int64_t monotonic_clock_ns, uint32_t frameId) {
        using Cmd = CommandType<decltype(&Driver::beginFrame)>::Command<&Driver::beginFrame>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.beginFrame_, std::move(monotonic_clock_ns), std::move(frameId));
    }

    inline backend::FenceHandle createFence(int dummy=0) {
        backend::FenceHandle result = mDriver.createFenceS();
        using Cmd = CommandType<decltype(&Driver::createFenceR)>::Command<&Driver::createFenceR>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.createFence_, backend::FenceHandle(result), std::move(dummy));
        return result;
    }

    inline backend::IndexBufferHandle createIndexBuffer(backend::ElementType elementType, uint32_t
indexCount, backend::BufferUsage usage) {
        backend::IndexBufferHandle result = mDriver.createIndexBufferS();
        using Cmd = CommandType<decltype(&Driver::createIndexBufferR)>::Command<&Driver::
createIndexBufferR>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.createIndexBuffer_, backend::IndexBufferHandle(result), std::move
(elementType), std::move(indexCount), std::move(usage));
        return result;
    }

    inline void terminate() {
        AutoExecute callOnExit([=]() {
        });
        return apply(&Driver::terminate, mDriver, std::forward_as_tuple());
    }

    inline backend::StreamHandle createStreamNative(void* stream) {
        AutoExecute callOnExit([=]() {
        });
        return apply(&Driver::createStreamNative, mDriver, std::forward_as_tuple(stream));
    }
...
};

```

实际上，我们可以看到，其实最终结果，就是在CommandStream中添加了一些对应DriverAPI方法的成员函数，而函数内部又用到了前面Driver.h内预处理生成的那些DriverAPI方法。

下面，我们继续看CommandStreamDispatcher.h部分：

```

//CommandStreamDispatcher.h

template<typename ConcreteDriver>
class ConcreteDispatcher {
public:

    static Dispatcher make() noexcept;

private:
//
#define DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params)
#define DECL_DRIVER_API(methodName, paramsDecl, params) \
    static void methodName(Driver& driver, CommandBase* base, intptr_t* next) { \
        SYSTRACE() \
        using Cmd = COMMAND_TYPE(methodName); \
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver); \
        Cmd::execute(&ConcreteDriver::methodName, concreteDriver, base, next); \
    }
#define DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) \
    static void methodName(Driver& driver, CommandBase* base, intptr_t* next) { \
        SYSTRACE() \
        using Cmd = COMMAND_TYPE(methodName##R); \
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver); \
        Cmd::execute(&ConcreteDriver::methodName##R, concreteDriver, base, next); \
    }
#include "private/backend/DriverAPI.inc"
};

//
template<typename ConcreteDriver>
UTILS_NOINLINE
Dispatcher ConcreteDispatcher<ConcreteDriver>::make() noexcept {
    Dispatcher dispatcher;

#define DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params)
#define DECL_DRIVER_API(methodName, paramsDecl, params) \
    dispatcher.methodName##_ = &ConcreteDispatcher::methodName;
#define DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) \
    dispatcher.methodName##_ = &ConcreteDispatcher::methodName;

#include "private/backend/DriverAPI.inc"

    return dispatcher;
}

```

在这里，我们看到使用DriverAPI.inc分成了两部分。我们先看第一部分：

DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params)同样被定义为空，在预处理过程会将DECL_DRIVER_API_SYNCHRONOUS声明的宏剔除。

DECL_DRIVER_API(methodName, paramsDecl, params)则被扩展为：

```

static void methodName(Driver& driver, CommandBase* base, intptr_t* next) {
    SYSTRACE()
    using Cmd = COMMAND_TYPE(methodName);
    ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
    Cmd::execute(&ConcreteDriver::methodName, concreteDriver, base, next);
}

```

这里的COMMAND_TYPE定义在CommandStream.h中：

```

//CommandStream.h

#define COMMAND_TYPE(method) CommandType<decltype(&Driver::method)>::Command<&Driver::method>

```

DECL_DRIVER_API(methodName, paramsDecl, params)最终变为：

```
static void methodName(Driver& driver, CommandBase* base, intptr_t* next) {
    SYSTRACE()
    using Cmd = CommandType<decltype(&Driver::methodName)>::Command<&Driver::methodName>
    ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
    Cmd::execute(&ConcreteDriver::methodName, concreteDriver, base, next);
}
```

而DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params)会被扩展为:

```
static void methodName(Driver& driver, CommandBase* base, intptr_t* next) {
    SYSTRACE()
    using Cmd = CommandType<decltype(&Driver::methodNameR)>::Command<&Driver::methodNameR>
    ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
    Cmd::execute(&ConcreteDriver::methodNameR, concreteDriver, base, next);
}
```

那么将DriverAPI.inc内API声明代入CommandStreamDispatcher中:

```
template<typename ConcreteDriver>
class ConcreteDispatcher
{
    static void flush(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::flush)>::Command<&Driver::flush>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::flush, concreteDriver, base, next);
    }

    static void beginFrame(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::beginFrame)>::Command<&Driver::beginFrame>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::beginFrame, concreteDriver, base, next);
    }

    static void createFence(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::createFenceR)>::Command<&Driver::createFenceR>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::createFenceR, concreteDriver, base, next);
    }

    static void createIndexBuffer(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::createIndexBufferR)>::Command<&Driver::
createIndexBufferR>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::createIndexBufferR, concreteDriver, base, next);
    }
};
```

我们看到, 在内部声明了一系列静态方法, 而方法内部调用过程主要是获取实体driver对象, 并调用Cmd的execute方法来实现对应的ConcreteDriver方法调用。

继续看第二部分, 这里DECL_DRIVER_API_SYNCHRONOUS(RetType, methodName, paramsDecl, params)又是空实现, 不再赘述。

DECL_DRIVER_API(methodName, paramsDecl, params)实现为:

```
dispatcher.methodName_ = &ConcreteDispatcher::methodName;
```

就是将dispatcher的methodName_和ConcreteDispatcher::methodName进行关联, 调用dispatcher的方法就是调用了ConcreteDispatcher的方法。

最后, DECL_DRIVER_API_RETURN(RetType, methodName, paramsDecl, params) 被扩展为:

```
dispatcher.methodName_ = &ConcreteDispatcher::methodName;
```

和DECL_DRIVER_API作用一样，只是处理的是带参数的API。

代入DriverAPI.inc的API接口声明后，实际代码扩展为：

```
//
template<typename ConcreteDriver>
UTILS_NOINLINE
Dispatcher ConcreteDispatcher<ConcreteDriver>::make() noexcept {
    Dispatcher dispatcher;
    dispatcher.flush_ = &ConcreteDispatcher::flush;
    dispatcher.beginFrame_ = &ConcreteDispatcher::beginFrame;

    dispatcher.createFence_ = &ConcreteDispatcher::createFence;
    dispatcher.createIndexBuffer_ = &ConcreteDispatcher::createIndexBuffer;
    return dispatcher;
}
```

可以看到，ConcreteDispatcher实际上就是针对DriverAPI的实现代理类，对Dispatcher调用最终都会转移为ConcreteDispatcher调用，并经由Cmd::execute调用，最终调用到DriverAPI上。

最后，对于NoopDriver.h，实现了一套空的DriverAPI，而VulkanDriver.h、MetalDriver.h、OpenGLDriver.h则是不同平台下实际驱动代码实现，在我们看来，他们都是对Driver.h中接口声明的最终实现。也就是我们最终调用的都是这些实现类的DriverAPI实现。由于它们的源文件中就有真正的函数实现，所以就不分析头文件中生成的函数了。

这些内容看起来麻烦，实际上就是用宏扩展来实现减少重复代码。